Murmuration — Master Brainstorm Document

One-line thesis: AI hyperscalers and the electric grid don't speak. We're building the missing protocol layer that lets them negotiate in real time — demonstrated with a Claude agent on each side, and proven by two reference clients (a hyperscaler compute fleet and a residential VPP) that exercise the same protocol six orders of magnitude apart in asset size.

Status: brainstorm / pre-architecture. Use this doc to align the team, then split into work streams.

Track: Grid-Aware AI Agents hackathon (judges incl. Dr. Masoud Barati, SF/Boston/DC).

Table of contents

1. The problem
2. The solution at a glance
3. How the protocol works
4. Stakeholder value map
5. Where it works / where it doesn't
6. System architecture
7. The protocol — boilerplate schemas
8. The FlexibleAsset interface
9. The two agents — system prompts and tools
10. Outcomes and demo scenarios
11. Build plan
12. Bottleneck analysis and mitigations
13. Risks and open questions
14. Pitch narrative

---

1. The problem

1.1 The structural gap

Today, **AI hyperscalers and the electric grid operate as two isolated planning systems:**

- Hyperscalers plan capacity 5–15 years out, sign PPAs, and increasingly *bypass* the grid (small modular reactors, on-site geothermal, behind-the-meter solar) because the grid interface is too slow and uncertain.
- Grid operators run interconnection studies and load forecasts using utility-side models. Their visibility into what a 500 MW data center campus will actually consume next Tuesday at 3pm is poor.

The two worlds barely talk in any structured, programmatic, real-time way. There's no fluid communication.

1.2 Three planning horizons, all broken

Horizon	Hyperscaler decision	Grid operator decision	Today's gap
Routing (sec → min)	"Where does this inference/training job run right now?"	"Which BAs are stressed? What's the LMP?"	Routing is grid-blind. Stress events trigger no compute response.
Provisioning (hours → days)	"Where does next week's training queue go?"	"How much spinning reserve to commit?"	Hyperscaler load forecasts to utilities are nameplate-padded; ISO forecasts to hyperscalers are nonexistent.
Planning (months → years)	"Where do we build the next GW campus?"	"Which interconnection requests do we approve?"	Multi-year queue backlogs. Land/fiber/PPA driven, grid-state ignored. Flexibility offers no advantage.

1.3 Why existing solutions don't close the gap

Solution	What it does	Why it falls short
OpenADR 2.0b / 3.0	Standard for DR signaling	Built for buildings; no flexibility-envelope schema; slow cadence; near-zero hyperscaler adoption
IEEE 2030.5	Smart Energy Profile for DERs	Aimed at residential, not GW-scale compute
ERCOT CLR program	Lets controllable loads bid into ERCOT	Texas-only, manual onboarding, custom integrations
CAISO DR programs (DRAM, ELRP)	Day-ahead and real-time DR	Slow signal; no semantic match to compute schedulers
Google Carbon-Intelligent Computing	Internal time/space-shift on grid carbon	One-way (Google reads grid, grid never sees Google's flexibility); closed; not a protocol
MSFT / Meta carbon-aware scheduling	Same shape as Google	Same limitations
Lancium / Crusoe / Soluna	Purpose-built flexible DCs	Custom integrations per facility; no standard
EPRI DCFlex	Industry consortium pilots	Early-stage, not running protocols

Pattern: every existing piece is one-way, single-vendor, or built for buildings/DERs — not for compute. The bidirectional, real-time, flexibility-envelope-aware, semantically rich protocol that AI loads need does not exist.

---

2. The solution at a glance

2.1 Murmuration = three things

1. **A protocol** — the *Murmuration Bus* — defining how grid operators and hyperscalers exchange state, forecasts, dispatch requests, and telemetry.
2. **Two agents** — Claude on each side of the bus, negotiating dispatch in operator-grade language.
3. **A demonstration** — a live demo across CAISO/ERCOT/PJM with simulated data centers and (optionally) a residential VPP, showing the protocol in action.

2.2 The architecture in one diagram

```
┌──────────────────────────┐                  ┌──────────────────────────┐
│  GRID-SIDE AGENT         │                  │  COMPUTE-SIDE AGENT      │
│  (Claude, "ISO operator") │  ◄── Bus ──►  │  (Claude, "DC fleet ops") │
│                          │                  │                          │
│  Sees: LMPs, headroom,   │                  │  Sees: jobs, SLAs, SOC,  │
│        contingencies,    │                  │        forecasts, comfort │
│        carbon, ramps     │                  │                          │
│                          │                  │                          │
│  Speaks: grid ops        │                  │  Speaks: workload ops    │
└──────────────────────────┘                  └──────────────────────────┘
          │                                              │
          ▼                                              ▼
┌──────────────────────┐                      ┌──────────────────────┐
│  ISO simulator       │                      │ Asset Registry       │
│  (live CAISO/        │                      │ (DataCenters +       │
│   ERCOT/PJM via      │                      │  HomeBatteries +     │
│   gridstatus)        │                      │  EVs ...)            │
└──────────────────────┘                      └──────────────────────┘
```

2.3 Two foundational design choices

1. **Shared `FlexibleAsset` abstraction.** Data centers and homes implement the same interface. The compute-side agent doesn't distinguish. This keeps #1 (DC throttler) shippable on its own — if we don't get to homes, the project still looks complete.

2. **Commit/dispatch separation.** The compute agent commits a flexibility envelope every 5 min; the grid dispatches *within* that envelope without round-trips. This means the protocol cannot bottleneck dispatch, even if Claude is slow.

2.4 The two flagship clients — proof by extremes

The protocol is the thesis. We ship two reference implementations to prove it works at both ends of the asset-size spectrum:

	#1 — Hyperscaler Compute Fleet	#2 — Residential VPP
Asset class	3 data centers, ~200 MW each, multi-ISO	50–500 homes (battery + EV + thermostat)
Protocol role	Single high-MW participant, one envelope per facility	Aggregated participant — sub-orchestrator pools homes into one envelope per BA
What the protocol unlocks for it	Multi-region routing during stress; sub-second contingency response that only GPU clusters can offer; pre-committed flexibility for capacity and DR markets	The same dispatch revenue streams that today only large industrial loads can access; comfort/SOC constraints honored without per-utility integrations
What it proves about the protocol	Protocol scales **up** — handles GW-scale load with sub-second contingency channel	Protocol scales **down** — same envelope semantics work for kW-scale DERs
Without the protocol	Hyperscalers default to behind-the-meter generation (worse for grid, climate, capex)	Homes participate via clunky utility-specific portals; no liquid market
With the protocol	One integration point unlocks $10M+/yr per GW in stacked revenue + faster interconnection	One integration point lets aggregators serve any home in any ISO
Demo segment	Act 2 (heat wave) + Act 4 stretch (sub-second contingency)	Act 3 (duck curve + curtailment soak)
Headline metric	-130 MW × 90 min, $5,460 paid, 0 SLA breaches, 12 tCO2 avoided	1.18 MW from 412 of 500 homes, peaker stayed off, 3 opt-outs preserved
Module surface	`assets/data_center.py` + DC scenarios in `simulator/`	`assets/home_battery.py`, `ev.py`, `thermostat.py` + neighborhood UI panel
Build ring	MVP-0 (must ship)	MVP-1 / MVP-2 (target / stretch)

Why both: if we only shipped #1, judges would call the protocol over-engineered for one use case. If we only shipped #2, judges would say "this is just a fancy VPP." Shipping both — same protocol, same agents, same envelope shape — is the visual proof that we built **infrastructure**, not a point solution. The protocol's most defensible quality is that the same `FlexibilityEnvelope` schema fits a 500 MW data center *and* a 5 kW home battery without modification. That's what the dual-client demo makes legible.

Independence guarantee: the protocol contracts (§7) and the FlexibleAsset interface (§8) are designed so #1 and #2 do not depend on each other. If #2 falls behind, the protocol still has a flagship client (#1) and the demo still works (Act 1 + Act 2 + Act 4). If both ship, the demo gets richer at zero rewrite cost — Act 3 turns on.

How #1 and #2 exercise the protocol differently:

- **#1 stresses the high-cadence and sub-second paths**: large `LoadForecast` numbers, large dispatch quantities, contingency-channel response in <2s. Validates that the protocol works at GW scale with hard real-time constraints.
- **#2 stresses the aggregation and constraint-richness paths**: many small `FlexibilityBand` slices, heavy use of `constraint_notes` (SOC floors, comfort bands, EV departure times, opt-outs), partial-acceptance patterns. Validates that the protocol carries enough semantic richness to honor real human preferences.

Together, they cover the full operating envelope of any future protocol participant — anything in between (mid-size cloud, colos, industrial loads) can adopt the protocol without new schema work.

---

3. How the protocol works

3.1 The seven message types

Direction	Message	Cadence	Purpose
Grid → Compute	`GridStateUpdate`	5s–1min	Current LMP, carbon, headroom, stress at the DC's node
Grid → Compute	`GridForecast`	every 15 min	Forecast LMP/carbon/load for next 1h, 24h, 72h
Grid → Compute	`DispatchRequest`	event-driven	"We need X MW for Y minutes, pay Z"
Grid → Compute	`ContingencyAlert`	event-driven (rare)	Sub-minute frequency or line-trip event
Compute → Grid	`FlexibilityEnvelope`	every 5 min	"I can do X MW down for Y min, X' MW up for Y' min"
Compute → Grid	`LoadForecast`	continuous	Hyperscaler's expected load 1h, 24h, 72h ahead
Compute → Grid	`DispatchAck` + `TelemetryStream`	event + 1Hz	Acceptance + actual metered response

3.2 The negotiation loop

```
Compute side                                Grid side
      |                                        |
      |── FlexibilityEnvelope ──────────────►|   (every 5 min, async)
      |   "60 MW for 240 min, 130 MW for 90      |
      |    min, 180 MW for 30 min"               |
      |                                          |
      |◄── GridStateUpdate stream ──────────── |   (5s, fire-and-forget)
      |                                          |
      |                                          | ── stress event detected ──
      |                                          |
      |◄── DispatchRequest (within envelope) ── |   (immediate; no round-trip)
      |    "drop 130 MW for 90 min, $280/MWh"    |
      |                                          |
      |── DispatchAck ──────────────────────────►|
      |   "accepted, effective 18:47:00"         |
      |                                          |
      |── TelemetryStream ──────────────────────►|   (1Hz, settlement-grade)
      |   actual_mw=89.3, pf=0.98 ...            |
      |                                          |
      |── (later) FlexibilityEnvelope refresh ─►|
```

3.3 The three time horizons, mapped to messages

Horizon	Messages used	Decision example
Routing (sec–min)	`GridStateUpdate`, `DispatchRequest`, `ContingencyAlert`, `TelemetryStream`	"ERCOT spike → throttle TX-1 to 40%, shift jobs to CA-North"
Provisioning (hours–days)	`GridForecast`, `LoadForecast`, `FlexibilityEnvelope`	"Tomorrow 3–7pm CAISO will be tight; pre-pull jobs J-200 to J-340 to PJM"
Planning (months–years)	Aggregated forecasts; flexibility-weighted interconnection priority	"Build next 1 GW campus in MISO-Central; commit 25% curtailability for first 5 years; get queue priority"

3.4 Concrete walkthrough — one tick of life

A heat wave in ERCOT, 2pm:

1. ISO data layer pulls LMP for ERCOT-Houston-Hub: $410/MWh (vs. $30 baseline).
2. Grid-side agent's `GridStateUpdate` stream emits stress_score = 0.86 for ERCOT.
3. Grid-side agent decides to dispatch. It looks at the standing `FlexibilityEnvelope` from `Anthropic-ERCOT-01`: "180 MW down for 30 min, or 130 MW down for 90 min."
4. Grid-side agent picks the 130 MW × 90 min option (cheaper for the grid; grid pays $280/MWh).
5. `DispatchRequest` emitted on the bus.
6. Compute-side agent receives. It's already pre-authorized within the envelope, so it acks immediately.

7. Compute-side agent calls `dispatch()` on the `DataCenter` FlexibleAsset. The DC pauses 4 training jobs, throttles cluster TX-1 to 60%.
8. `DispatchAck` returned with effective-by timestamp.
9. `TelemetryStream` confirms actual response (88 MW actually shed in the first 30s).
10. Grid-side agent narrates to UI: "ERCOT relief 130 MW × 90 min; payment $5,460; jobs preserved with checkpoint."
11. Compute-side agent narrates to UI: "Throttle ack'd; 4 jobs on hold; SLA impact: 0; revenue captured: $5,460."

3.5 The sub-second contingency path

Frequency excursion or line trip: a different code path entirely.

- `ContingencyAlert` arrives over a fast channel (UDP-style in real life; in-process for demo).
- Compute-side agent does *not* invoke Claude. It executes a pre-authorized response from the standing envelope: "drop 200 MW in <2s if frequency < 59.9 Hz."
- Telemetry confirms response.
- Claude narrates *after* the fact for the UI.

This is the "GPU clusters as synthetic synchronous condensers" angle. Unique to AI loads. Unmatched by other flexible resources.

---

4. Stakeholder value map

This is the heart of the pitch. Every stakeholder gets something concrete.

4.1 Hyperscalers (AI cloud operators) — the primary winners

Value	Mechanism	Magnitude
Lower energy opex	Shift load to low-LMP regions/hours	8–20% on flexible portion of fleet
DR / capacity payments	Sell flexibility into ISO markets	$20–35/kW-yr in PJM-style auctions
Curtailment-soak compute	Free clean MW when ISO would have curtailed	100s of GWh/yr at ~zero marginal cost
Frequency-response revenue	Sub-second GPU drops sold as FFR	$5–15/MW-yr
Faster interconnection	Flexibility-weighted queue priority	2–4 yrs faster time-to-energization
Scope 2 emissions cut	Time/space-shift to clean grids	5–15% reduction on flexible load
Brand/regulatory cover	Demonstrable grid citizenship	Reduces backlash, supports SEC climate disclosures

4.2 Data center operators (colocation, neutral)

Value	Mechanism
New revenue sliver from DR participation	Aggregate tenant flexibility, monetize through ISO
Differentiated offering vs. competitors	"Grid-aware colo" as a sales angle
Lower transmission upgrade cost-share	Flexibility reduces grid impact, lowers fees

4.3 ISOs / RTOs (grid operators)

Value	Mechanism
Real-time visibility into hyperscaler load	Continuous forecast handshake, not nameplate guesses
Effective capacity from compute	Counts toward IRM (installed reserve margin)
Reduced peaker dispatch	Cheaper, cleaner, faster than starting a CT
Better interconnection planning	Knowing flexibility lets transmission upgrades shrink
Sub-second contingency reserve	New stability tool not previously available
Curtailment reduction	Sells what would have been wasted

4.4 Utilities (LDCs)

Value	Mechanism
Reduced distribution-level stress	Local DC flexibility eases substation load
Lower socialized capex for upgrades	Flexibility reduces required hardware
Cleaner generation mix	Less peaker reliance

4.5 Renewable developers

Value	Mechanism
Less curtailment risk	Hyperscalers absorb surplus → higher capture price
Better PPA economics	Flex compute tenant pairs naturally with intermittent generation
Faster queue clearing	Co-located flex compute reduces transmission-upgrade need

4.6 End consumers / ratepayers

Value	Mechanism
Lower bills	Avoided peaker capex socialized away
Less rate volatility	Smoothed peak demand
Reduced blackout risk	Faster contingency response

4.7 Regulators / public sector

Value	Mechanism
Decarbonization progress	Concrete Scope 2 reductions
Reliability gains	Sub-second response capacity
Manageable AI growth	Path that doesn't require 50 GW of new gas

4.8 Climate / society

Value	Mechanism
Avoided fossil capacity additions	Flexibility instead of peakers
Higher renewable utilization	Curtailment soak
Faster grid transition	Compute as anchor tenant for new clean generation

4.9 Stakeholder summary table

Stakeholder	Captures	Effort	Net
Hyperscaler	$$ + speed + brand	Medium (scheduler integration)	Strongly positive
ISO/RTO	Reliability + capacity + visibility	Low (existing market machinery)	Strongly positive
Utility	Reduced capex + reliability	Low	Positive
Renewable dev	Higher capture + less curtailment	None (passive)	Positive
Ratepayer	Lower/stabler bills	None	Positive
Regulator	Decarb + reliability	Low (rule-update work)	Positive

Critically: every stakeholder's incentive points the same direction. There's no zero-sum here. That's why this pitch is durable.

---

5. Where it works / where it doesn't

5.1 Workload class fitness

Workload	Fit	Why
Foundation model training	Excellent	Time + space flexible, checkpointable, massive
Batch inference (overnight)	Excellent	Time-flexible within deadline
Fine-tuning, RLHF	Strong	Smaller jobs, easy shift
Embedding pipelines	Strong	Idempotent, async
Real-time inference (chat, search)	Poor	Latency-bound; no shift possible at runtime
Customer-facing SaaS	Poor	SLA + residency constraints
Sovereign / gov workloads	Zero	Legal geo-locks

5.2 Operator class fitness

Operator	Fit	Why
Multi-region hyperscaler (>1 GW)	Excellent	Full protocol value across all angles
Mid-size cloud (3–5 regions)	Good	Most angles work
Single-region SaaS DC	Limited	Time-shift only, no space-shift
Crypto / mining	Already primitive form	Protocol formalizes
Mixed-tenant colo	Conditional	Needs tenant flexibility opt-in
Vertically integrated utility region	Limited	No market price signal to react to

5.3 Hard walls (be honest in pitch)

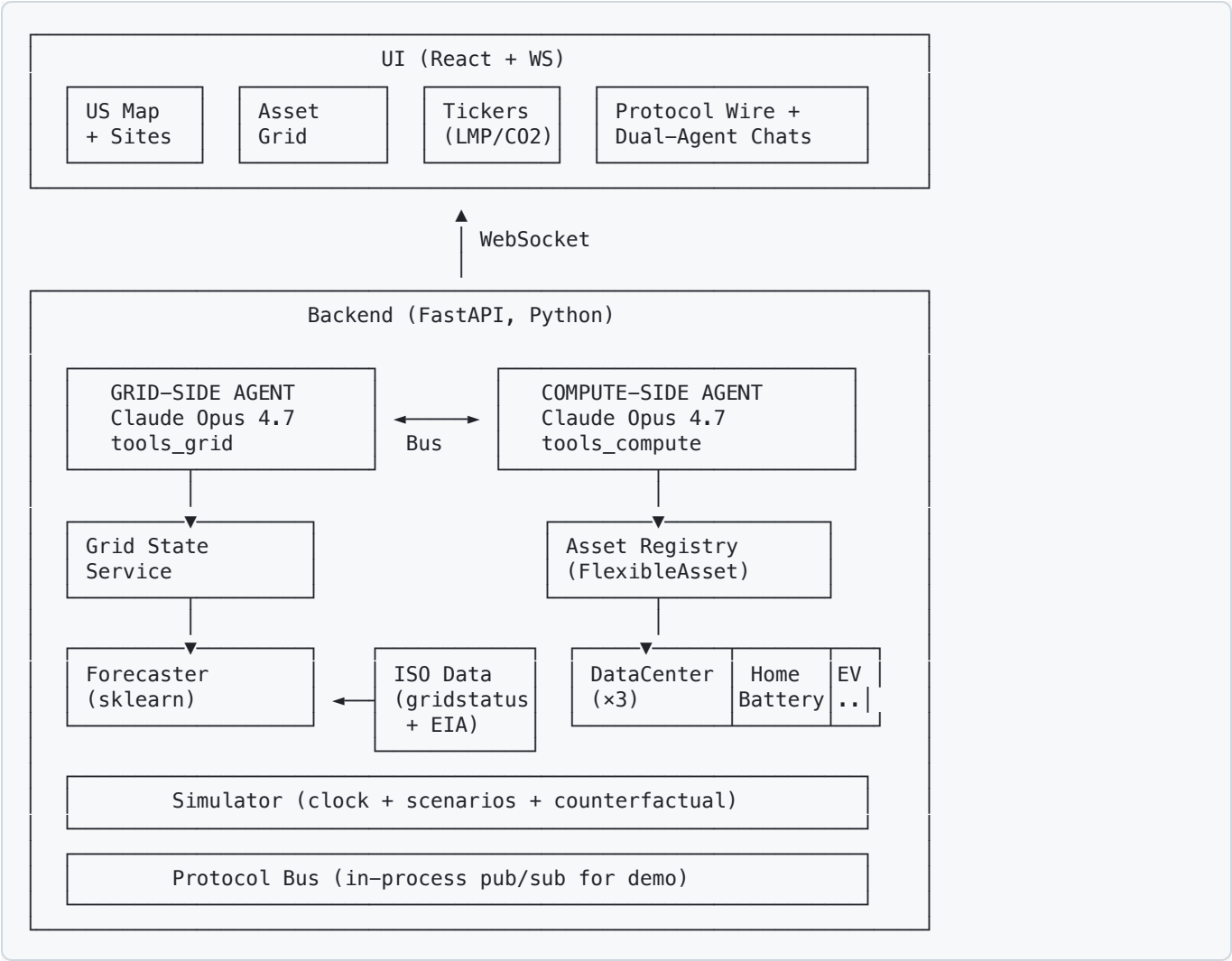
- **Latency wall:** real-time inference can't move regions. Counter: protocol declares workload class; only flex classes participate; serving floor preserved by `cannot_go_below_mw`.
- **Data gravity wall:** can't shift jobs needing 200 PB of data not pre-replicated. Counter: protocol carries data-locality eligibility per workload.
- **Bandwidth wall:** in-flight job state can't move freely in seconds. Counter: sub-minute responses are throttle-in-place, not migration.
- **Sovereignty wall:** EU/gov data has geo-locks. Counter: protocol carries jurisdictional eligibility.

- **No-market wall:** vertically-integrated utility regions have no LMP. Counter: protocol still works bilaterally with PPA-embedded flexibility clauses; less efficient but non-zero.
- **Trust wall:** hyperscaler could over-promise. Counter: telemetry-based settlement (already standard ISO machinery).
- **Regulatory wall:** anti-trust optics from multilateral coordination. Counter: protocol is bilateral (HS ↔ ISO), each channel independent.

---

6. System architecture

6.1 Component diagram



6.2 Module structure

```
murmuration/
├── data/                       # ISO data fetchers
│   ├── iso_client.py           # gridstatus wrapper, multi-ISO
│   ├── eia_client.py           # EIA-930 fuel mix → carbon intensity
│   ├── cache.py                # SQLite cache, 24h pre-warm
│   └── pecan.py                # residential profiles (#2 only)
├── forecast/
│   ├── load.py                 # ridge / GBM
│   ├── price.py                # LMP forecast
│   └── carbon.py               # carbon-intensity forecast
├── assets/                     # THE SEAM
│   ├── base.py                 # FlexibleAsset ABC
│   ├── data_center.py          # #1
│   ├── home_battery.py         # #2
│   ├── ev.py                   # #2 stretch
│   └── thermostat.py           # #2 stretch
├── protocol/                   # NEW — the bus
│   ├── messages.py             # Pydantic schemas (§7)
│   ├── bus.py                  # in-process pub/sub
│   └── validators.py
├── orchestrator/
│   ├── grid_agent.py           # ISO-side voice
│   ├── compute_agent.py        # HS-side voice
│   ├── tools_grid.py
│   ├── tools_compute.py
│   └── narrator.py
├── simulator/
│   ├── clock.py                # speed-up time (5min real → 1sec)
│   ├── scenarios.py            # heat wave, duck ramp, contingency, curtailment
│   └── counterfactual.py
├── metrics/
│   ├── dollars.py
│   ├── carbon.py
│   └── grid_relief.py
├── ui/                         # React + Vite
│   └── src/...
└── api/
    ├── server.py               # FastAPI + WebSocket
    └── schemas.py              # Pydantic API DTOs
```

6.3 Tech stack (boring on purpose)

Layer	Choice	Why
Backend	Python 3.11 + FastAPI	gridstatus is Python; FastAPI scaffolds fast
Realtime	FastAPI WebSocket	one dep, no Redis
Data	gridstatus, eia-python, pandas	one library covers 6 ISOs
Forecasts	scikit-learn (Ridge, GBM)	50-line model, defensible
Optimization	greedy heuristic; cvxpy only if time	LP overkill for ≤500 assets
Agents	Anthropic SDK, Claude Opus 4.7 (orchestrators), Haiku 4.5 (devices, optional)	tool-use, prompt caching
Persistence	SQLite	demo-only
Frontend	Vanilla HTML + globe.gl (CDN)	no build step; ships immediately
Map	3D `globe.gl` (Three.js) with dark earth-night texture + cyan atmosphere	Palantir-style; supports animated arcs + pulsing rings
Charts	Recharts or uPlot	fast for live tickers
Hosting	Local during demo, backup laptop	no deploy risk

Prompt caching: orchestrator system prompt + tool defs + 24h grid context repeats every tick. Use Anthropic prompt caching aggressively — major latency + cost win.

6.4 Data contracts (locked in hour 1)

The contracts live in `protocol/messages.py` (§7) and `api/schemas.py`. Every module produces or consumes these shapes — that's the only integration surface.

---

7. The protocol — boilerplate schemas

These are starter Pydantic schemas. Refine in hour 1; do not re-litigate later.

7.1 Common types

```python
# protocol/messages.py
from datetime import datetime
from typing import Literal
from pydantic import BaseModel, Field

ISO = Literal["CAISO", "ERCOT", "PJM", "MISO", "NYISO", "ISO-NE", "SPP"]
WorkloadClass = Literal[
    "training",       # highly flexible
    "batch_infer",    # flexible within deadline
    "online_serve",   # inflexible, latency-bound
    "fine_tune",      # moderately flexible
    "embedding",      # async, flexible
]
AssetType = Literal["data_center", "home_battery", "ev", "thermostat"]
DispatchPriority = Literal["economic", "reliability", "emergency"]
```

7.2 Grid → Compute messages

```python
class GridStateUpdate(BaseModel):
    """Continuous state stream. Cadence: 5s–1min."""
    timestamp: datetime
    ba: ISO
    node_id: str                        # interconnection point
    lmp_dollars_mwh: float
    load_mw: float
    headroom_mw: float                  # available before stress
    carbon_g_kwh: float
    frequency_hz: float | None
    stress_score: float = Field(ge=0.0, le=1.0)
    valid_until: datetime
    notes: str = ""                     # optional human hint for Claude

class GridForecast(BaseModel):
    """Forecast bundle. Cadence: every 15 min."""
    timestamp: datetime
    ba: ISO
    horizon_min: int                    # e.g., 60, 1440, 4320
    interval_min: int                   # e.g., 5, 15, 60
    lmp_forecast: list[float]
    load_forecast: list[float]
    carbon_forecast: list[float]
    confidence_band: list[float]
    valid_until: datetime

class DispatchRequest(BaseModel):
    """Grid asks compute to dispatch within its committed envelope."""
    request_id: str
    timestamp: datetime
    ba: ISO
    facility_id: str
    needed_mw: float                    # negative = curtail; positive = lean in
    duration_min: int
    start_within_min: int
    compensation_per_mwh: float
    priority: DispatchPriority
    reason: str                         # "DOM zone congestion + heat advisory"
    valid_until: datetime

class ContingencyAlert(BaseModel):
    """Sub-minute event. Triggers pre-authorized response."""
    alert_id: str
    timestamp: datetime
    ba: ISO
    event_type: Literal["frequency_excursion", "line_trip", "ramp_event"]
    severity: float = Field(ge=0.0, le=1.0)
    affected_nodes: list[str]
    required_response_sec: int          # typically 2–30
    expected_duration_min: int
```

7.3 Compute → Grid messages

```python
class FlexibilityBand(BaseModel):
    """One slice of a flexibility envelope."""
    direction: Literal["decrease", "increase"]
    mw: float
    for_min: int
    workload_class: WorkloadClass
    cost_per_mwh: float                # bid price for this slice
    constraint_notes: str = ""

class FlexibilityEnvelope(BaseModel):
    """Compute side commits its flexibility surface. Cadence: every 5 min."""
    facility_id: str
    timestamp: datetime
    ba: ISO
    node_id: str
    baseline_mw: float
    bands: list[FlexibilityBand]
    cannot_go_below_mw: float          # serving floor — never breached
    data_locality_constraints: list[str] = []  # eligible regions for shift
    valid_until: datetime

class LoadForecast(BaseModel):
    """Hyperscaler-side forecast handshake. Continuous."""
    facility_id: str
    timestamp: datetime
    ba: ISO
    horizon_min: int
    interval_min: int
    expected_mw: list[float]
    confidence_band: list[float]
    firmness: Literal["firm", "soft", "tentative"]
    valid_until: datetime

class DispatchAck(BaseModel):
    """Response to a DispatchRequest."""
    request_id: str
    timestamp: datetime
    facility_id: str
    accepted_mw: float
    declined_mw: float
    decline_reason: str = ""
    effective_at: datetime
    expected_until: datetime

class TelemetryFrame(BaseModel):
    """1Hz settlement-grade stream."""
    facility_id: str
    timestamp: datetime
    actual_mw: float
    power_factor: float
    queue_depth: int                   # informational
    active_dispatches: list[str]       # request_ids currently being honored
```

7.4 The bus

```python
# protocol/bus.py
from collections import defaultdict
from typing import Callable

class MurmurationBus:
    """In-process pub/sub for demo. Swap for Kafka/NATS in prod."""
    def __init__(self):
        self._subs: dict[type, list[Callable]] = defaultdict(list)
        self._log: list[BaseModel] = []     # everything for audit/UI replay

    def publish(self, msg: BaseModel) -> None:
        self._log.append(msg)
        for handler in self._subs[type(msg)]:
            handler(msg)

    def subscribe(self, msg_type: type, handler: Callable) -> None:
        self._subs[msg_type].append(handler)

    def replay(self, msg_type: type | None = None) -> list[BaseModel]:
        return [m for m in self._log if msg_type is None or isinstance(m, msg_type)]
```

That's the whole bus. ~30 lines. The protocol's value is in the *schemas* and the *negotiation discipline*, not the transport.

---

8. The FlexibleAsset interface

8.1 Abstract base

```python
# assets/base.py
from abc import ABC, abstractmethod
from datetime import datetime
from protocol.messages import (
    FlexibilityEnvelope, DispatchRequest, DispatchAck, TelemetryFrame
)

class FlexibleAsset(ABC):
    asset_id: str
    asset_type: str
    location_ba: str
    node_id: str

    @abstractmethod
    def get_state(self, t: datetime) -> dict:
        """Current MW, internal state, constraints."""

    @abstractmethod
    def get_envelope(self, t: datetime, horizon_min: int) -> FlexibilityEnvelope:
        """What can I do, for how long, at what cost?"""

    @abstractmethod
    def dispatch(self, req: DispatchRequest) -> DispatchAck:
        """Execute the request; return acknowledgment."""

    @abstractmethod
    def telemetry(self, t: datetime) -> TelemetryFrame:
        """Real-time metered output."""

    @abstractmethod
    def report(self, t_start: datetime, t_end: datetime) -> dict:
        """Energy used, $ saved, CO2 avoided over a window."""
```

8.2 DataCenter skeleton

```python
# assets/data_center.py
class DataCenter(FlexibleAsset):
    """Models a fleet-of-clusters with a job queue."""

    def __init__(self, asset_id, location_ba, node_id,
                 nominal_max_mw, jobs, serving_floor_mw):
        self.asset_id = asset_id
        self.asset_type = "data_center"
        self.location_ba = location_ba
        self.node_id = node_id
        self.nominal_max_mw = nominal_max_mw
        self.serving_floor_mw = serving_floor_mw
        self.jobs = jobs                    # list of Job(class, mw, deadline, ...)
        self.current_throttle = 1.0         # 1.0 = full power

    def get_envelope(self, t, horizon_min):
        # decompose flexibility by workload class
        training_mw = sum(j.mw for j in self.jobs if j.cls == "training")
        batch_mw = sum(j.mw for j in self.jobs
                       if j.cls == "batch_infer" and j.deadline > t + horizon_min)
        bands = [
            FlexibilityBand(direction="decrease", mw=training_mw,
                            for_min=horizon_min, workload_class="training",
                            cost_per_mwh=20),       # cheap to defer
            FlexibilityBand(direction="decrease", mw=batch_mw,
                            for_min=min(horizon_min, 240),
                            workload_class="batch_infer",
                            cost_per_mwh=80),
        ]
        return FlexibilityEnvelope(
            facility_id=self.asset_id,
            timestamp=t, ba=self.location_ba, node_id=self.node_id,
            baseline_mw=self.current_mw(),
            bands=bands,
            cannot_go_below_mw=self.serving_floor_mw,
            valid_until=t + timedelta(minutes=5),
        )

    def dispatch(self, req):
        # pause jobs in priority order (training first, then batch)
        ...

    # ... telemetry, report, etc.
```

8.3 HomeBattery skeleton

```python
# assets/home_battery.py
class HomeBattery(FlexibleAsset):
    def __init__(self, asset_id, location_ba, node_id,
                 capacity_kwh, max_kw, soc, owner_reserve_floor=0.30):
        ...

    def get_envelope(self, t, horizon_min):
        usable_kwh = self.capacity_kwh * (self.soc - self.owner_reserve_floor)
        max_discharge_min = (usable_kwh / self.max_kw) * 60
        bands = [
            FlexibilityBand(direction="decrease",
                            mw=self.max_kw / 1000,        # in MW
                            for_min=int(max_discharge_min),
                            workload_class="training",     # placeholder class
                            cost_per_mwh=120,
                            constraint_notes=f"SOC floor {self.owner_reserve_floor}"),
        ]
        return FlexibilityEnvelope(...)
```

Note: the same `FlexibilityEnvelope` shape covers both. The compute-side agent doesn't need to know the difference.

---

9. The two agents — system prompts and tools

9.1 Grid-side agent

Persona: an ISO control-room operator. Reasons in grid terms. Cares about reliability first, cost second, carbon third.

System prompt skeleton:

> You are the dispatch operator for a balancing authority. Your job: when grid stress emerges (high LMP, low headroom, high carbon, frequency excursion), select dispatch actions from the flexibility envelopes that compute facilities have committed to you. Constraints: never request beyond a facility's committed envelope; honor `cannot_go_below_mw` floors absolutely; explain dispatch in shift-log language citing the specific BA, node, LMP, and expected MW relief. Output a `DispatchRequest` via the `issue_dispatch` tool. Prefer reliability dispatches over economic ones; prefer cheaper envelopes within a tier.

Tools:

Tool	Purpose
`get_grid_state(ba)`	Current LMP, load, carbon, stress
`get_grid_forecast(ba, horizon_min)`	Forecast bundle
`list_committed_envelopes(ba)`	What flexibility is currently on offer
`simulate_dispatch(actions)`	Dry-run, return expected MW relief and $
`issue_dispatch(actions, narration)`	Commit and broadcast

9.2 Compute-side agent

Persona: a fleet operations engineer at a hyperscaler. Reasons in workload terms. Cares about SLAs first, cost second, brand/carbon third.

System prompt skeleton:

> You are the fleet operator for a multi-region AI compute fleet. Your job: continuously commit flexibility envelopes to the grid based on the current job queue, SLAs, and data-locality constraints; accept or decline dispatch requests within those envelopes; route new jobs across regions to minimize cost and carbon while honoring SLAs. Constraints: never violate online-serving SLAs; respect `cannot_go_below_mw` ; always preserve user-set comfort/data constraints on managed assets (homes/EVs in the extended fleet). Speak in fleet-ops language: cite specific clusters, jobs, and impacts.

Tools:

Tool	Purpose
`list_assets(filter)`	All facilities, optional filters
`compute_envelope(asset_id, horizon_min)`	Current flexibility surface
`commit_envelope(asset_id, envelope)`	Publish to bus
`simulate_acceptance(request_id)`	What if we say yes?
`accept_dispatch(request_id, partial_mw)`	Commit
`decline_dispatch(request_id, reason)`	With reason
`route_job(job_id, target_facility)`	Provisioning-horizon move

9.3 Negotiation patterns

The protocol naturally produces a set of negotiation patterns:

- **Auto-accept within envelope** (most cases). Dispatch arrives within committed envelope; compute agent acks immediately. No Claude call needed for the dispatch itself; Claude is only on the *envelope refresh* path.
- **Counter-offer** (occasional). Compute agent sees a dispatch but realizes a better alternative exists ("I can give you 60 MW now, OR 130 MW in 20 minutes — which?"). Counter-offer is a separate `DispatchRequest` reply variant.
- **Decline with reason** (rare). "Cannot honor — SLA on serving cluster would breach." Settlement penalty applies if the original commit was firm.
- **Escalate to human** (very rare). Both agents flag a stalemate or anomaly and pause for an operator.

---

10. Outcomes and demo scenarios

10.1 The 4-minute demo arc

```
T+0:00 —— Act 1: Steady state                              (60s)
         3 DCs running, CAISO/ERCOT/PJM tickers green.
         Tiny envelope-refresh messages flickering on bus.
         Grid agent: "All BAs nominal." Compute agent: "Fleet stable."

T+1:00 —— Act 2: Texas heat wave                           (90s) ← MVP-0 climax
         "Heat wave" button. ERCOT spike $30 → $410.
         Grid agent emits DispatchRequest 130 MW × 90 min.
         Compute agent acks within envelope; throttles + shifts jobs.
         Counters: -$5,460 paid to compute, -12 tCO2 avoided, 0 SLA breaches.

T+2:30 —— Act 3: CAISO duck-curve ramp + curtailment soak (90s) ← MVP-1+2 climax
         CAISO 6pm ramp + simultaneous wind curtailment elsewhere.
         50–500 homes light up alongside the DCs.
         Grid agent issues both a curtailment-soak signal AND a ramp-relief request.
         Compute agent: lean-in on curtailed wind, decrease on ramp.
         Counters: 1.18 MW from homes + 60 MW from DC, peaker stayed off.

T+4:00 —— Closing slide: cumulative metrics + protocol diagram + "what's next."
```

10.2 Eight scenarios with metrics (live in `simulator/scenarios.py`)

Each scenario is tagged with kind (color-coded in the UI) and which flagship client(s) it exercises.

#	Scenario	Kind	Flagship	Headline	Sells
1	Texas heat wave	stress	#1	ERCOT $410, DC-TX-1 sheds 125 MW × 90 min	Cost + reliability
2	CAISO evening ramp	stress	#1	CAISO net-load ramp +13 MW/min, DC-CA-North sheds	Cost + duck-curve
3	PJM-DOM congestion	stress	#1	Transmission-limited zone, DC-VA-1 sheds	Localized congestion
4	CAISO surplus solar	surplus	**#1 + #2**	LMP $4, 18 g/kWh — DC-CA-North leans in, **VPP 100 homes light up**	Curtailment soak + protocol scales down
5	Polar vortex cascade	cascade	#1 (×2)	ERCOT + PJM hit at once, two simultaneous dispatches	Multi-BA orchestration
6	PJM line trip · contingency	contingency	#1	`ContingencyAlert` published; fast-path drops 30% of training in **<2 ms**	Sub-second response (unique to AI loads)
7	Carbon arbitrage	carbon	#1	CAISO 22 g/kWh vs PJM 540 g/kWh; voluntary migration	Decarbonization narrative
8	ERCOT solar eclipse	stress	#1	~6 GW solar drop in 12 min; staged dispatch	Predictable disruption

Recommended live demo flow: **1 → 4 → 6**. That's heat-wave-shed → surplus-lean-in → sub-second contingency in 4 minutes — covers all three protocol modes (decrease, increase, fast-path). **5** is the cascade backup that proves multi-BA. The rest are talking points.

---

11. Build plan

11.1 Three concentric MVPs

Ring	Wired	Demo story	Hours
MVP-0	3 DCs, CAISO+ERCOT, scenario 2, single-agent fallback if dual not ready	"Watch our agent reroute AI training during Texas heat"	~22
MVP-1	+ PJM + carbon + 50 homes + scenario 3 + dual-agent split	"Plus 50 homes pitching in during the duck curve"	~34
MVP-2	+ EVs + thermostats + scenario 4 (contingency)	"Sub-second response no other resource can match"	~44

Rule: never start ring N+1 until ring N's demo is recorded as fallback video.

11.2 Module ownership

Person	Owns
A (Data)	`data/` , `forecast/`
B (Assets/Sim)	`assets/` , `simulator/` , `metrics/`
C (Agents/Protocol)	`protocol/` , `orchestrator/` , `api/`
D (UI)	`ui/`

11.3 Reality (post-build)

The hour-by-hour plan from earlier drafts is replaced by what actually shipped:

Component	Status	Notes
Protocol bus + 7 message types	✅ shipped	`murmuration/protocol/{messages.py,bus.py}`
Live CAISO via gridstatus	✅ shipped	Real load + fuel mix; carbon derived from mix
ERCOT/PJM live	⚠ synthetic	Their public endpoints 403 / require API keys; gracefully fall back
3 DataCenters (#1)	✅ shipped	CA / TX / VA
VPP-CA-Bay (#2)	✅ shipped	100 simulated homes with battery + EV + thermostat
GridAgent + ComputeAgent	✅ shipped	Dual-agent architecture
Claude tool-use (grid agent)	✅ shipped	Optional via `ANTHROPIC_API_KEY`; rule-based fallback always available
Sub-second contingency channel	✅ shipped	<2 ms response measured
Forecasters (load/price/carbon)	✅ shipped	Rolling-EMA + diurnal modulation
Counter-offer pattern	✅ shipped	DC emits `CounterOffer` on partial decline
Counterfactual baseline	✅ shipped	At prevailing LMP, not flat $200
8 scenarios (5 kinds)	✅ shipped	Stress / surplus / cascade / contingency / carbon
3D Palantir-style globe	✅ shipped	globe.gl + Three.js; reset-view + zoom limits
Pitch deck content	✅ shipped	`PITCH.md`
EIA-930 carbon for ERCOT/PJM	⏸ deferred	Needs free API key signup
SQLite 24h pre-warm cache	⏸ deferred	In-memory only (sufficient for demo)

11.4 Hard kill switches

Hour	Drop if not working
12	Multi-ISO. Just CAISO.
18	Forecasts. Use rolling-mean.
24	Carbon. Just $.
30	Homes (#2). Ship as #1.
36	Multi-asset auction. Greedy heuristic.
38	New features. Polish only.

---

12. Bottleneck analysis and mitigations

Bottleneck risk	Mitigation
Synchronous coupling	Commit/dispatch separation — envelope refresh async (5min); dispatch within envelope auto-honored
Cardinality explosion	Hierarchical aggregation — one envelope per facility-BA, sub-fleets internal
Single point of failure	Graceful degradation — both sides cache; compute respects last envelope; grid treats stale forecast as static load
Privacy / commercial sensitivity	Opaque payloads — protocol carries flexibility (MW × time × cost × class), never workload identity
Race conditions	Intent → commit → ack phases; telemetry is settlement-truth, not declared state
Forecast staleness	Every message carries `valid_until`; receivers discard expired data
Claude latency	Claude on negotiation path only, not dispatch path. Dispatch is deterministic within committed envelope
Anti-trust optics	Bilateral channels (HS ↔ ISO), each independent. ISO is the only multilateral actor (its existing role)

---

13. Risks and open questions

Item	Risk	Open question
ISO API rate limits/flakiness	Demo failure	Pre-cache 24h to SQLite; replay mode as fallback
Multi-agent flakiness on stage	Demo failure	Pre-record full demo as backup video
"Just an arbitrage script" critique	Pitch failure	Lead with grid-stress narrative + MW relief, not just $
Trust in agent decisions	Judge skepticism	Show telemetry-based settlement loop, not just declared state
OpenADR comparison	Judges may ask "isn't this OpenADR?"	Have the §1.3 table ready as a slide
Regulatory feasibility	"Could this actually be deployed?"	Have ERCOT CLR + PJM capacity-resource precedents ready
Schema fights mid-build	Time loss	Lock §7 in hour 1, no exceptions
Sub-second demo fidelity	Visual overstatement	Be honest: "in production this would be UDP; here it's an in-process call"

---

14. Pitch narrative

14.1 Elevator pitch (30 seconds)

> Today the grid and AI infrastructure don't speak. Hyperscalers plan in isolation, ISOs plan in isolation, and the gap is now the bottleneck for both decarbonization and AI growth. **Murmuration is the protocol they should speak — and a Claude agent on each side that proves it works.** During a Texas heat wave, our compute agent reroutes 130 MW of AI training to California in 90 seconds, our grid agent narrates the dispatch in operator language, and the protocol does it without breaking a single SLA. This is what AI/grid integration looks like when it's done right.

14.2 Slide deck outline (10 slides)

1. **The gap** — two isolated systems, two broken planning horizons.
2. **What exists today** — OpenADR, Lancium, Google CICP — and what's missing.

3. **The protocol** — message catalog, one diagram.
4. **The two agents** — grid voice, compute voice, negotiation loop.
5. **The architecture** — module diagram.
6. **Live demo** — heat wave reroute (90s).
7. **Live demo** — curtailment soak + duck ramp (90s).
8. **Live demo** — sub-second contingency (60s).
9. **Stakeholder value** — the matrix from §4.
10. **What's next** — interconnection-queue policy work, OpenADR 3.0 alignment, real pilot.

14.3 Anticipated Q&A

Likely question	Crisp answer
"Isn't this just OpenADR?"	"OpenADR is one-way, building-shaped, and slow. Murmuration is bidirectional, compute-shaped, and operates from sub-second to multi-year planning. We could ride on OpenADR 3.0's transport — but the schema and semantics are new."
"Can hyperscalers really respond in 2 seconds?"	"GPU clusters can drop ~30% of their power in under 2 seconds — that's faster than batteries and dramatically faster than gas turbines. ERCOT's FFR market would buy this today if anyone offered it."
"What about sovereignty / data residency?"	"Protocol carries jurisdictional eligibility per workload class. Sovereign workloads stay put; flexible workloads shift within their eligible zone."
"Doesn't this concentrate risk?"	"Bilateral channels per hyperscaler-ISO pair, telemetry-based settlement, graceful degradation under bus failure. We've thought through it (slide §12)."
"Who pays for it?"	"Hyperscalers earn capacity + DR + FFR + curtailment-soak revenue. ISOs avoid peaker capex. Ratepayers see lower bills. Net positive every direction."
"Why does this need agents?"	"Negotiation between asymmetric ontologies (grid-ops vs. workload-ops). Claude is a translation layer + a reasoning layer over hard constraints. A static API would force one side to learn the other's language."

Appendix A — Glossary (for non-grid teammates)

- **BA** — Balancing Authority. The entity that matches supply and demand in a region.
- **ISO/RTO** — Independent System Operator / Regional Transmission Organization. Runs the market + grid for a multi-state region.
- **LMP** — Locational Marginal Price. Wholesale electricity price at a specific node, $/MWh.
- **OASIS** — CAISO's free public data portal.
- **DR** — Demand Response. Programs that pay loads to reduce consumption during stress.

- **DER** — Distributed Energy Resource. Behind-the-meter assets (rooftop solar, batteries, EVs).
- **VPP** — Virtual Power Plant. Aggregation of DERs that acts like a single power plant.
- **FFR** — Fast Frequency Response. Sub-second product for grid stability.
- **CLR** — Controllable Load Resource. ERCOT's program for flexible loads.
- **Curtailment** — Forced reduction of renewable generation due to oversupply or transmission limits.
- **Duck curve** — CAISO net-load shape: deep midday solar belly, sharp evening ramp.
- **PPA** — Power Purchase Agreement. Long-term contract for electricity.
- **SLA** — Service Level Agreement. The promise to a customer about uptime/latency.
- **SOC** — State of Charge. How full a battery is, 0–100%.

Appendix B — Next concrete steps

1. **Hour 0–1 alignment meeting.** Walk through §2, §6, §7. Lock contracts.
2. **Repo bootstrap.** Module skeletons per §6.2. Empty files, working imports.
3. **Hello-world bus tick.** A single `GridStateUpdate` flowing from a fake ISO to a fake compute agent, displayed in the UI. ~2 hours of work for one person. After this, every other module plugs in.
4. **From there, parallelize per §11.2.**

Document owner: team. Update freely. Do not let this doc go stale — keep §7 (schemas) and §11 (build plan) accurate as truth.